Northeastern University, Khoury College of Computer Science

CS 6120 Natural Language Processing
Project Proposal
March 30, 2026

Lingyun Xiao, Linxuan Li, Sherwin Vahidimowlavi, Zongyang Li
`xiaolingyun412, LinxuanLi-DS, sherwinvahidi, zongyang078`

xiao.ling@northeastern.edu, li.linxuan@northeastern.edu, vahidimowlavi.s@northeastern.edu, li.zongyang@northeastern.edu

1 Proposed Objective

This project develops **CruxBot**, a Retrieval-Augmented Generation (RAG) system that serves as an intelligent rock climbing assistant. CruxBot consolidates fragmented climbing knowledge—route databases, community discussions, accident reports, and technique guides—into a unified retrieval system powered by a locally hosted LLM, enabling climbers to receive accurate, context-aware answers with clickable citations to original source passages.

2 Background and Motivation

Following the Heilmeier Catechism framework:
What are we trying to do? Build a domain-specific RAG system that transforms scattered climbing information into an accessible, conversational knowledge base. Climbing is a rapidly growing sport with over 35 million active participants worldwide, yet reliable information remains fragmented across forums, wikis, and niche websites.
How is it done today, and what are the limits? Climbers rely on keyword search across Mountain Project, Reddit (r/climbing, r/bouldering, r/climbharder), and scattered blog posts. This has key limitations: (1) keyword search cannot parse nuanced queries like *"I climb V4 with weak finger strength—how should I train?"*; (2) useful advice is buried in thousands of forum replies; (3) no unified interface spans multiple sources; (4) safety-critical information (accident reports, protection beta) is difficult to surface.
What is new in our approach? CruxBot applies semantic vector retrieval over a curated corpus of 500,000+ climbing documents from six sources and uses a locally hosted LLM to synthesize coherent, cited answers. Unlike generic chatbots, every response includes clickable

citations to the original passage, and metadata filters allow content-type-aware retrieval (e.g., prioritizing accident reports for safety queries).

Who cares? Climbers of all levels—beginners seeking gear advice, intermediate climbers designing training plans, and advanced climbers planning outdoor trips. A future iteration could serve as a production tool embedded in climbing community websites, reducing the barrier to accessing high-quality climbing knowledge.

Related work. RAG was introduced by Lewis et al. [1] as a cost-effective alternative to fine-tuning for knowledge-intensive tasks. Domain-specific RAG systems such as BioASQ (biomedical QA) and FinGPT (financial NLP) demonstrate that curated domain corpora significantly reduce hallucination. We apply this paradigm to climbing, a domain not yet explored in the literature.

Dataset. We have collected and cleaned data from six sources totaling 338,433 unified entries after deduplication:

Source	Content	Method	Entries
OpenBeta [2]	Routes, grades, GPS (47 US states)	GraphQL API (BFS)	85,898
Mountain Project (Kaggle) [3]	Routes with descriptions & URLs	Kaggle download	116,700
MP Forums (Kaggle)	Training, gear, technique discussions	Kaggle download	99,173
AAC Articles [4]	Accident reports, expedition records	Kaggle download	27,828
Reddit	Recent community discussions (2024–2026)	Public JSON API	2,372
Gear Reviews	Equipment reviews & ratings	Kaggle download	6,462
Total (after deduplication and cleaning)			**338,433**

OpenBeta and Kaggle MP routes had 76.6% name overlap; we kept the Kaggle version (richer descriptions and clickable URLs) and supplemented with OpenBeta-only routes. All entries were cleaned (HTML removal, DMCA text filtering, short-text removal) and normalized into a unified 12-field schema. 100% of entries have clickable source URLs for citation. The retrieval challenge lies in the heterogeneity of sources—structured route data vs. conversational forum posts vs. long-form accident reports—requiring robust chunking and embedding strategies.

3 Proposed Approach / Implementation Details

Our pipeline follows **Ingest** → **Normalize** → **Chunk** → **Embed** → **Retrieve** → **Generate**:

- **Pre-processing:** All six sources are normalized into a unified schema: `doc_id, title, text, content_type, source, source_url, grade, route_type, location, lat, lng, metadata`. Route data lacking descriptions (OpenBeta: 0.6% have text) is enriched by composing structured fields into natural language (e.g., *"Moonlight Buttress is a 5.12d trad route in Utah ¿ Zion"*). Forum posts and articles undergo HTML removal, whitespace normalization, and minimum-length filtering.

- **Retrieval algorithm:** Dense vector retrieval with cosine similarity over embeddings generated by `all-MiniLM-L6-v2` (384-dim), stored in ChromaDB [5]. Metadata filters (`content_type`, climbing type, grade range, location) narrow the search space before similarity ranking. Top-5 chunks are retrieved per query. We will also explore hybrid search (BM25 + dense) if pure dense retrieval underperforms on keyword-heavy queries.

- **Noise remediation:** Forum data is filtered by minimum text length (50 chars) and upvote thresholds. Duplicates across OpenBeta and Kaggle MP are resolved via name

matching (94,511 merged). DMCA complaint text is detected and removed (7 entries). Retrieved chunks are re-ranked by relevance score before being passed to the LLM.

- **Scale of data:** 338k documents are chunked into an estimated 400k–500k chunks. ChromaDB handles this efficiently with HNSW approximate nearest neighbor search. Embeddings are pre-computed offline; only the query embedding is computed at inference time.

- **Design decisions:** Chunk size of 300–500 tokens balances context preservation with retrieval granularity. 50-token overlap prevents information loss at boundaries. Short documents (<500 tokens, e.g., individual route entries) remain as single chunks; long documents (AAC articles avg. 2,685 chars, long forum threads) are split. Each chunk retains the parent document's metadata (`source_url`, `content_type`) for citation tracing. We will experiment with chunk sizes (200/300/500) and top-k values (3/5/10).

- **Validation and evaluation:** (1) Retrieval quality: Precision@5 and Recall@5 on 50+ manually annotated query-document pairs across five content types; (2) Generation quality: human evaluation of factual accuracy, relevance, and citation correctness on a 1–5 Likert scale; (3) Baseline comparison: RAG-augmented responses vs. vanilla LLM responses (no retrieval) to quantify improvement; (4) Latency: target <5s end-to-end, with profiling of embedding, retrieval, and generation stages; (5) Content-type coverage: verify that the system can answer route, training, safety, and gear queries with appropriate source types.

- **LLM:** Llama 3 8B (or Mistral 7B as fallback), served locally via Ollama [6] on GCP—no external API calls. 4-bit quantization (GGUF format) to fit within GPU memory constraints, with streaming output for perceived responsiveness.

Frontend and deployment. A Streamlit chat interface provides the user-facing application. Each response displays clickable citation links to the original source passage. Users can filter by content type (routes, training, safety, gear). The entire solution is containerized with Docker for reproducibility, with a `README.md` and `docker-compose.yml` for one-command setup.

```
Query → Streamlit → Embed → ChromaDB (Top-K) → Ollama/LLM → Cited Response
```

References

[1] Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS*.
[2] OpenBeta GraphQL API. https://api.openbeta.io (CC0 License)
[3] Mountain Project Dataset (Kaggle). https://www.kaggle.com/datasets/pdegner/mountain-project-rotues-and-forums
[4] AAC Articles Dataset (Kaggle). https://www.kaggle.com/datasets/iantonopoulos/american-alpine-club-articles (CC0)
[5] ChromaDB. https://docs.trychroma.com/
[6] Ollama. https://ollama.ai/